

TECHNICAL WHITEPAPER · CREDITCOIN CC3 TESTNET

Faction March

*An Attestcoin-Secured Cross-Chain Territory
War, and an Undercollateralized Credit Line
Backed by Proof, Not Collateral*

Track: Gaming (primary) · DeFi (secondary)

Built for: BUIDL CTC 2026 Fall — Attestcoin Protocol theme

Chains: Ethereum Sepolia (order origination) · Creditcoin CC3 Testnet
(resolution, credit)

Status: Live on testnet · 91 passing Foundry tests · 9 adversarial findings
addressed

ABSTRACT

Cross-chain applications typically resolve the question “what happened on chain A?” by asking a trusted intermediary — a bridge, a relayer, an oracle committee — and accepting its answer on faith. That intermediary is consistently the largest attack surface in cross-chain systems, independent of how secure the underlying chains are. The Attestcoin Protocol removes the intermediary for one specific class of claim: transaction inclusion. A block-prover precompile lets a contract on Creditcoin CC3 cryptographically verify that a specific transaction was included in a specific Ethereum Sepolia block, with no relayer's word required.

Faction March is built to answer a question a toy demonstration cannot: does this primitive hold up when real value and real adversaries are attached to it? Combat orders in a three-faction territory war are Sepolia transactions; they take effect on Creditcoin only once a cryptographic proof of their inclusion is submitted and verified on-chain, in the same transaction that resolves the fight. A second contract, *WarChest*, demonstrates a related claim in decentralized finance: that a faction's cryptographically-provable territory can serve directly as a creditworthiness signal, enabling undercollateralized lending without a trusted appraiser or locked collateral. This paper describes the protocol, its security model, its economic model, and the testing methodology used to validate both against live testnet state rather than local simulation alone.

1. Motivation

Nearly every cross-chain application in production today depends on some party — a multisig, a light-client relayer network, a validator set distinct from either chain — to attest to state on a foreign chain. Because that attestation is itself unverifiable by the receiving chain without trusting the attester, it becomes the practical security ceiling of the entire system, regardless of how secure the two underlying chains are individually. This is not a hypothetical risk: bridge and relayer compromises constitute one of the largest categories of loss in decentralized finance, consistently larger than exploits of the settlement chains they connect.

The Attestcoin Protocol's block-prover precompile is a narrower, more verifiable primitive: given a `ContinuityResponse`, it cryptographically confirms that a specific `txBytes` was included at a specific `txIndex` in a specific Sepolia block, and that the containing block chains back to a checkpoint Creditcoin has already attested. No party's word is taken on faith — the guarantee is purely cryptographic. This is deliberately narrow: it proves inclusion, and nothing about the intent behind a transaction, or which contract's business logic should care about it. That gap is exactly what a consuming contract must close correctly, and closing it correctly under real adversarial pressure — not a benign demo — is the first problem this project addresses.

The second, related problem is economic. Undercollateralized lending in decentralized finance is rare because there is usually no cheap, trustworthy signal of a borrower's creditworthiness on-chain; the default solution is to require collateral worth more than the loan, which is capital-inefficient and excludes anyone without idle capital to lock. *WarChest* explores a narrow instance of an alternative: when a borrower's standing is itself the output of a cryptographically-verified process — here, territory captured only through attested combat — that standing can be trusted directly, with no separate collateral posted.

2. System Architecture

Faction March spans two blockchains and one off-chain relay role, with no centralized backend holding any authoritative game state. Order origination happens on Ethereum Sepolia; verification, combat resolution, and credit all happen on Creditcoin CC3. A single off-chain component, the *courier*, carries a cryptographic proof between the two chains — it has no privileged authority and anyone may act as one, including a player proving their own order.

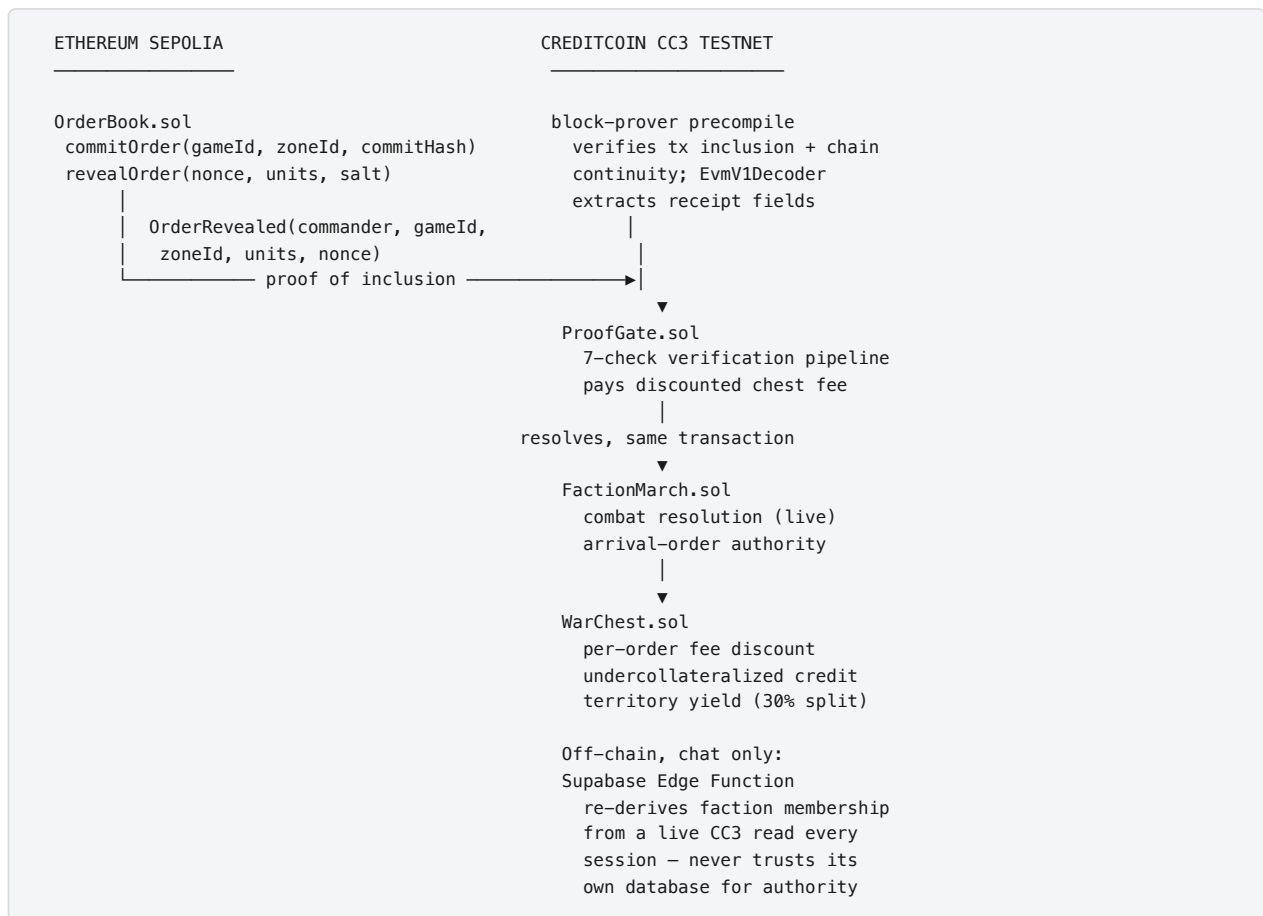


Figure 1. Two-chain data flow. Value moves only within each chain; only a proof of inclusion crosses between them.

No bridge moves value between the chains. Ether spent placing an order stays on Sepolia (collected by `OrderBook`'s immutable treasury); CTC spent proving an order and running the game economy stays on Creditcoin. What crosses the chain boundary is exclusively a cryptographic proof of Sepolia state, never an asset.

3. Protocol Mechanics

3.1 Commit-reveal order placement

A design tension sits at the center of the game: hiding unit counts is strategically valuable (an opponent who cannot see incoming force size cannot optimally reinforce against it), but combat resolves continuously throughout a game, live, the instant a proof lands — not in one batch at game end. Hiding orders until settlement would force every order into a single end-of-game resolution pass, which is a fundamentally different game.

The resolution adopted hides units only for the interval that matters: between commitment and reveal. `OrderBook.commitOrder(gameId, zoneId, commitHash)` locks in a target zone and pays a flat ETH fee against `commitHash = keccak256(abi.encode(units, salt))`, exposing the zone but nothing about army size. `revealOrder(nonce, units, salt)` — callable only by the original committer, and only once — later exposes the real values once they hash back to the stored commitment, emitting `OrderRevealed`, the event the remainder of the pipeline proves against. The commander controls reveal timing entirely; nothing reveals automatically. Two limits are accepted rather than concealed: reveal calldata is visible in Sepolia's public mempool before it is mined, and a zone's garrison is never secret once resolved — only the unresolved order leading up to it is.

3.2 Proof verification — the seven checks

`ProofGate.submitOrderProof` (and its batched counterpart, §3.4) is the sole trust boundary between the two chains. After the block-prover precompile confirms inclusion and `EvmV1Decoder` extracts receipt fields, `ProofGate` runs seven checks, in order, each with a dedicated custom error:

#	CHECK	ERROR	PURPOSE
1	Emitter allowlist	<code>ForgedEmitter</code>	Log address must equal the deployed <code>OrderBook</code> , fixed at deploy.
2	Topico match	<code>WrongTopic0</code>	Must equal <code>keccak256("OrderRevealed(address,uint256,uint16,uint32,uint64)")</code> .
3	Topic count	<code>WrongTopicCount</code>	Guards against an unrelated event colliding on the same topico with a different topic arity.
4	Live gameId binding	<code>GameNotActive</code>	Checked against <code>FactionMarch.currentState</code> so an order cannot resolve against a foreign or inactive game.
5	Exact replay guard	<code>OrderAlreadyProcessed</code>	Keyed on <code>keccak256(blockHeight, txIndex, logIndex)</code> — not tx hash or block alone, since two orders may share a block.
6	Ordering cursor	<i>deliberately absent</i>	A monotonic per-game cursor would reject exactly the out-of-order proof arrivals the game depends on (§3.3).
7	Staleness window	<code>OrderStale</code>	1,200 Sepolia blocks (≈ 4 hours), checked against the <code>ChainInfo</code> precompile, so a hoarded proof cannot surface in a later, unrelated game.

Each check has a dedicated negative unit test. Verification and combat resolution occur in a single transaction: once all seven checks pass, `ProofGate` calls `FactionMarch.resolveOrder` directly — there is no separate settlement step for a third party to front-run or censor.

3.3 Arrival-order authority

Combat resolves deterministically the instant a proof lands: an attacker exceeding the defending garrison flips the zone, survivors becoming the new garrison; an attacker falling short shrinks the garrison instead. When two orders target the same zone, priority is given to whichever *proof* reaches Creditcoin first — not whichever order was sent first on Sepolia. This is a deliberate design choice, not an artifact: sorting by Sepolia send time would erase the timing-race mechanic the courier economy (§3.4, §5) is built around. It is verified directly rather than merely asserted — two tests submit the same pair of competing orders' proofs in opposite sequences and confirm opposite winners.

3.4 Bountied, permissionless batching

Proof submission carries no privileged role. Any address may submit any valid proof, earning a fixed bounty (`BOUNTY_PER_ORDER = 0.0001 CTC`) for doing so; a commander may courier and prove their own order. `submitOrderProofBatch` lands up to ten proofs (`MAX_BATCH_SIZE = 10`, the native precompile's own limit) sharing a single continuity proof in one Creditcoin transaction. Measured directly against real transaction receipts — not a local gas estimate — a ten-order batch costs 56.9% less gas than ten independent submissions (1,695,981 gas actual versus 3,938,060 projected). A single invalid order reverts the entire batch, which incentivizes couriers to bundle only orders they are already confident will succeed. Two independent couriers racing for the same bounty resolve to exactly one payout; the loser's transaction reverts cleanly.

4. Economic Model — WarChest

`WarChest` reads territory from `FactionMarch` as a pure, read-only consumer and converts it into three independent forms of leverage, all keyed by `gameId` so no game inherits state from any other.

4.1 Fee discount

Every successful proof submission requires the courier to attach a native CTC fee (`CHEST_FEE_PER_ORDER = 0.00005` CTC), deposited into that order's game's chest in the same transaction. That fee is discounted according to the order's own commander's current faction territory, read live at the moment of proof — not snapshotted at game start, and not tied to the courier's own faction, since a courier frequently proves someone else's order.

ZONES HELD	DISCOUNT
≥ 3	5%
≥ 6	10%
≥ 9	20%

Because the tier is read after combat resolves within the same call, a batch that captures a faction's third zone partway through can already apply that discount to later orders in the same batch — the tier reflects truth as of execution, not as of submission.

4.2 Undercollateralized credit

`creditLimit = zonesHeld × BASE_CREDIT_PER_ZONE ( 0.001` CTC per zone), adjusted by a reputation multiplier: `+2%` (`REPAY_BONUS_BPS`) per completed repayment, and a permanent `-30%` (`DEFAULT_PENALTY_BPS`) per lifetime default that survives even after the debt is cleared. No collateral is posted separately; territory that can only exist because it was proven through the pipeline in §3 is the collateral. A single draw is capped at `20%` (`MAX_DRAW_BPS_OF_CHEST`) of the chest's current balance, so one faction cannot drain the shared pool and lock the others out. A draw not repaid within `REPAYMENT_WINDOW_BLOCKS` (5,000 blocks) of being taken defaults automatically — computed from block number, with no transaction required to trigger the default.

4.3 Territory yield

Every inflow to the chest — a manual top-up or a courier's proof fee — immediately splits `YIELD_SHARE_BPS = 30%` of that specific inflow across the three factions in proportion to zones currently held, credited to a per-faction `claimableYield` balance any member of that faction may withdraw. The remaining 70% funds the shared `chestBalance` backing credit limits. Splitting is performed at the instant funds arrive, against a snapshot of *current* ownership, rather than at claim time against a retrospective average: this makes capturing a zone moments before a large deposit unprofitable for that specific deposit, since a faction only ever earns a share of CTC that arrives after it already held the ground. Debt repayments are exempt from a second split, since that CTC already passed through the split once, when it first entered the chest.

DESIGN NOTE

These three mechanisms are deliberately non-overlapping in what they reward: the fee discount lowers the cost of continuing to play, the credit line converts territory into spendable capital with real default risk, and the yield share is a direct, riskless payout funded by the game's own activity rather than newly minted value. A faction can pursue any combination of the three from the same underlying signal — zones held.

5. Security Model

The system's security posture is documented against nine specific adversarial scenarios in the project's `SECURITY.md`, each resolved as Mitigated (with a named passing test), Accepted (an explicit, bounded limitation), or Inherited (a property assumed from the underlying protocol): forged emitter, event topic collision, exact and same-block replay, cross-game proof injection, stale proof resurrection, Sepolia reorganizations, courier censorship or no-show, bounty-race front-running, and mass cheap-order griefing.

One structural finding is worth surfacing directly: an early implementation of batched, per-order fee discounting deposited each order's fee as it was individually resolved, within a single pass. An underpaid batch could drain the attached `msg.value` against early orders and then hard-revert with no decodable error data on a later order's deposit call — before ever reaching the intended, legible `IncorrectChestFee` check. The fix restructures the batch entry point into two passes: resolve every order and sum required fees first, with zero value transferred; validate the attached value against that exact total once; only then execute every deposit in a second pass. This is a general pattern worth stating plainly — a batch operation that both computes and spends value per-item in a single pass can fail in a way that obscures the actual cause, and splitting compute-then-commit into two explicit passes is the fix, not a narrower guard clause.

A second, related finding surfaced later, in live play rather than in testing: the fee discount described in §4.1 is deliberately computed *after* `resolveOrder` executes, so an order that itself captures the zone crossing a discount threshold already earns that discount on itself, within the same transaction. A caller can only estimate the required fee from state read *before* sending that transaction, and so can never reliably predict a value that depends on the transaction's own side effects — the gap is not an implementation bug but a structural property of computing a fee this way. Requiring an exact `msg.value` match against that unpredictable total meant a harmless overpayment — the honest result of a conservative pre-flight estimate — reverted the entire submission rather than merely costing slightly more than strictly necessary. Both entry points now accept `msg.value` at or above the true required total and refund the excess to the caller as the final transfer in the call; only genuine underpayment still reverts with `IncorrectChestFee`. The general lesson generalizes past this one fee: whenever a contract's exact charge depends on state the charge-paying transaction itself will change, an exact-match payment check is the wrong primitive, and a floor-plus-refund is the right one.

6. Testing and Live Verification

91 Foundry tests pass across the project's two Foundry workspaces (Sepolia's `OrderBook.t.sol` and Creditcoin's `FactionMarch.t.sol`, `WarChest.t.sol`, `ProofGate.t.sol`). Every one of the seven checks in §3.2 carries its own dedicated negative test. Gas figures cited anywhere in the project's documentation, including §3.4's batching measurement, are drawn from real `cast receipt` data against live transactions rather than local estimates — Foundry's mocked block-prover precompile is a trivial, always-true stub whose cost does not scale with continuity-proof size the way the real precompile's does, so a local gas report alone would understate the batching benefit.

Beyond the unit-test suite, each major mechanic has been independently verified against live testnet state: arrival-order authority, the fee-discount calculation, commit-reveal round trips, and the faction-chat access-control boundary were each confirmed by driving the deployed frontend through a real wallet (a mocked EIP-1193 provider bridged to a funded `ethers.Wallet`) against the actual deployed contracts, not a local fork. This distinction is treated as material throughout the project: a passing unit-test suite demonstrates that the code behaves as written against a model of the environment, while live verification demonstrates that the model itself was correct.

7. Faction Chat: Off-Chain State Without Off-Chain Trust

Faction March includes one off-chain component with no on-chain equivalent: a per-`(gameId, faction)` chat channel backed by Supabase. Its access-control design is worth including here because it generalizes: an off-chain service is given no independent authority over a fact the chain already knows. The backing table, `faction_messages`, has row-level security enabled with zero grants to either the anonymous or authenticated Supabase roles — the table is unreachable through the public REST API under any condition, confirmed directly (a request with the anonymous key returns a permission-denied error, not an empty result, rather than being assumed from the policy configuration). The only path in is a single Edge Function that verifies an EIP-191 signature over a login challenge, then re-derives the caller's faction by reading `FactionMarch.commanderFaction` live from Creditcoin CC3 — never from anything Supabase has cached or been told previously — before minting a short-lived, HMAC-signed session scoped to exactly the value that call returns. A validly-signed request from a wallet that has not joined any faction is rejected with a live 403, not assumed to fail from the code alone.

8. Deployment

Contracts are deployed in a fixed dependency order: `FactionMarch`, then `WarChest` (which reads `FactionMarch` at construction), then `ProofGate` (constructed with both addresses). A one-shot `setProofGate` call, restricted to the original deployer and callable exactly once, wires `resolveOrder`'s access control on `FactionMarch` and `WarChest` to the deployed `ProofGate` address; until it is called, both fail closed. This is a narrow setup step rather than a standing administrative key — it moves no army, captures no zone, and cannot be redirected once set.

CONTRACT	NETWORK	ADDRESS
<code>OrderBook</code>	Ethereum Sepolia	<code>0xa9842871a176feeA29590de1A71DE829940FfC36</code>
<code>FactionMarch</code>	Creditcoin CC3	<code>0xba618275A71ea261cAbA7294e742589723aEC1AE</code>
<code>WarChest</code>	Creditcoin CC3	<code>0xcd3A69f231c93f37A2A7f61D5B5F58850850a045</code>
<code>ProofGate</code>	Creditcoin CC3	<code>0x8ff40DBA12240379e431eD7a927D16413717F6B8</code>

Complete superseded-address history and a phase-by-phase build log are maintained in the project repository's `README.md` and `spikes/FINDINGS.md`.

9. Conclusion

Faction March demonstrates that Attestcoin's inclusion-proof primitive can support a live, adversarial, real-value application beyond illustrative demonstration — and that doing so surfaces concrete engineering requirements (replay keys bound to log index rather than transaction hash, staleness windows, two-pass batch settlement, emitter allowlisting) that a lower-stakes proof of concept would not have forced. Separately, `WarChest` offers a small, testable instance of a broader claim: where an on-chain fact is provably unforgeable, it can substitute directly for posted collateral. Both claims are backed here by a passing test suite and by verification against real, live testnet transactions, not simulation alone.